

1. Address these comments
2. Add the SRS and HRS that the other project had as well to have a detailed report of the systems and their following capabilities.

Ranial-Systems-BMS-project

Battery Management System using BQ76952 and MCU communication.

1. Device Description

1.1 Device Requirements

This project is a **Battery Management System (BMS)** designed to monitor and protect a rechargeable battery pack. A battery pack is made up of multiple battery cells connected in series, and each cell must stay within a safe voltage, current, and temperature range. If any cell goes too high, too low, gets too hot, or the pack draws too much current, the battery can become damaged or unsafe. The purpose of this device is to detect those conditions early and respond before they become serious problems.

The system uses the **BQ76952 battery monitor IC** to measure important battery information such as individual cell voltages, pack current, and temperature. The BQ76952 can support battery packs with **3 to 16 cells in series**, making it useful for medium-sized battery systems. The battery current is measured using a **shunt resistor** placed in the negative path of the battery pack. As current flows through the shunt resistor, a small voltage is created across it. The BQ76952 reads this small voltage and uses it to calculate the current flowing into or out of the battery.

The BMS communicates with a microcontroller using **I2C communication**. I2C is a simple communication method that lets the microcontroller request data from the BQ76952 and send configuration commands to it. Through I2C, the microcontroller can read cell voltages, current measurements, temperature readings, protection flags, and device status information.

This device also includes several important protection features. It can detect unsafe conditions such as:

- Overvoltage, when a cell voltage becomes too high
- Undervoltage, when a cell voltage becomes too low
- Overcurrent, when too much current flows through the battery
- Short circuit conditions
- Overtemperature or undertemperature conditions
- Internal diagnostic faults

Some safety responses are handled directly by the BQ76952 because they are time-critical. For example, short circuit and overcurrent protection need to happen very quickly, so the battery monitor IC can react without waiting for the microcontroller. Other functions, such as reading values, displaying information, logging data, or adjusting safety limits, can be handled by firmware on the microcontroller.

The system is also designed to measure voltage accurately with a small margin of error. Since battery safety depends on knowing the true voltage of each cell, the firmware should include voltage thresholds with tolerance margins. For example, instead of only checking whether a cell is exactly above or below a limit, the code should compare the measured voltage against safe upper and lower boundaries. This helps prevent false readings from noise or small measurement errors.

Another feature of the BMS is **cell balancing**. Over time, cells in a battery pack may not stay at the exact same voltage. Some cells may charge slightly faster or discharge slightly slower than others. Cell balancing helps

reduce these differences so that the battery pack stays healthier and performs more consistently.

Overall, this project combines hardware monitoring and firmware control to create a safer and smarter battery pack. The BQ76952 handles accurate measurement and fast protection, while the microcontroller manages communication, decision-making, and user-level control. The final goal is to build a system that can reliably monitor battery health, detect dangerous conditions, and protect the battery pack during charging and discharging.

Simple Summary

In simple terms, this project acts like a safety guard for a battery pack. It constantly checks the battery cells, current, and temperature. If something unsafe happens, such as a cell voltage getting too high or too much current being drawn, the system can detect it and help shut the battery down before damage occurs.

Appendix: Requirements Specification

1. Software Requirements Specification

1.1 Overview

The software for this project shall control and monitor a Battery Management System using a microcontroller and the BQ76952 battery monitor IC. The software shall communicate with the BQ76952 using I2C, read battery measurements, check for unsafe operating conditions, and respond to faults. The software shall help protect the battery pack by monitoring cell voltage, pack current, temperature, and device status.

1.2 Users

There needs to be only one User Persona. Its not something that we can differentiate between Hardware and software.

The users of this software are students, engineers, and technicians who need to monitor the condition of a rechargeable battery pack. The user does not directly control the battery cells but can use the system to view battery status, detect faults, and verify that the battery pack is operating safely.

1.3 Definitions and Abbreviations

- **BMS:** Battery Management System
- **MCU:** Microcontroller Unit
- **IC:** Integrated Circuit
- **I2C:** Inter-Integrated Circuit, a communication protocol used between the MCU and BQ76952
- **ADC:** Analog-to-Digital Converter
- **SOC:** State of Charge
- **OV:** Overvoltage
- **UV:** Undervoltage
- **OCD:** Overcurrent in Discharge
- **OCC:** Overcurrent in Charge
- **SCD:** Short Circuit in Discharge
- **Shunt Resistor:** A small resistor used to measure current by sensing voltage drop

1.4 Functionality

Every Software Requirement will have a section of code that will be written to achieve it.

SRS 01 – The software shall communicate with the BQ76952 battery monitor IC using the I2C protocol.

- SRS 02** – The software shall read the voltage of each connected battery cell from the BQ76952.
- SRS 03** – The software shall compare each cell voltage to a programmed minimum and maximum safe voltage threshold. [For a 4s 12V battery what could be the voltages? \(write the thresholds\)](#)
- SRS 04** – The software shall identify an undervoltage fault when any cell voltage falls below the programmed minimum voltage threshold. [\(write the thresholds\)*](#)
- SRS 05** – The software shall identify an overvoltage fault when any cell voltage rises above the programmed maximum voltage threshold. [\(write the thresholds\)*](#)
- SRS 06** – The software shall include a voltage tolerance margin to reduce false fault detection caused by electrical noise or small measurement errors. [How is the software doing this? More detail required. Refer codes. What is supposed to happen once a false fault is detected?](#)
- SRS 07** – The software shall read battery pack current using the shunt voltage measurement provided by the BQ76952.
- SRS 08** – The software shall identify an overcurrent fault when the measured current exceeds the programmed safe current limit. [\(write the thresholds\)*](#)
- SRS 09** – The software shall read temperature data from the BQ76952 or connected temperature sensors.
- SRS 10** – The software shall identify a temperature fault when the measured temperature is outside the programmed safe operating range. [\(write the thresholds\) what is the temperature range?](#)
- SRS 11** – The software shall read protection status registers from the BQ76952 to determine if a fault condition has occurred. [Mention the different statuses for protection and Fault conditions](#)
- SRS 12** – The software shall report battery status information, including cell voltages, current, temperature, and fault states. [Report where exactly? display/ serial monitor?](#)
- SRS 13** – The software shall support fault handling by disabling or warning against unsafe charging or discharging conditions. [what part of the software? Details on the parts of the software codes that would do this work and how?](#)
- SRS 14** – The software shall initialize the BQ76952 before normal battery monitoring begins.
- SRS 15** – The software should support cell balancing control to reduce voltage differences between cells. [Clarity on what these "thresholds" are](#)
- SRS 16** – The software should allow safety thresholds to be adjusted in firmware for testing and calibration.
- SRS 17** – The software shall not ignore critical fault conditions reported by the BQ76952. [how? what action does it take](#)
- SRS 18** – The software shall not allow normal battery operation to continue if a serious protection fault is active. ["The MCU will send a STOP signal to the battery in case of a serious protection fault"](#)
[* In case of overcharge the MCU will read the data from ___ and send a stop signal to stop the charging of the battery](#)
[* In case of High temperature, the MCU reads data from ___ and sends a ___ signal via ___ protocol to ___ to do ___\(action Item\)](#)
[This way of writing clearly will give us clearer understanding of the project.](#)

2. Hardware Requirements Specification

2.1 Overview

The hardware for this project shall use the BQ76952 battery monitor IC to monitor and protect a multi-cell rechargeable battery pack. The hardware shall support voltage sensing, current sensing, temperature sensing, communication with a microcontroller, and battery protection features. The system shall be designed to detect unsafe battery conditions and protect the battery pack from damage.

2.2 Definitions and Abbreviations

- **BMS:** Battery Management System
- **BQ76952:** Battery monitor and protection IC used in this project
- **MCU:** Microcontroller Unit
- **I2C:** Inter-Integrated Circuit communication protocol
- **NFET:** N-channel Field Effect Transistor
- **ADC:** Analog-to-Digital Converter
- **Cell:** One individual battery unit inside a larger battery pack
- **Series Connection:** Battery cells connected so their voltages add together
- **Shunt Resistor:** A precision resistor used to measure current
- **Thermistor:** A temperature-sensitive resistor used for temperature measurement

2.3 Functionality

HRS 01 – The hardware shall use the BQ76952 battery monitor IC as the main battery monitoring device.

HRS 02 – The hardware shall support monitoring of a battery pack containing 3 to 16 series-connected cells.

HRS 03 – The hardware shall provide individual cell voltage connections from the battery pack to the BQ76952. [The HRS 09 should be mentioned before you talk about the things that are done via that I2C connection.](#)

HRS 04 – The hardware shall allow the BQ76952 to measure each individual cell voltage.

HRS 05 – The hardware shall include a shunt resistor in the negative current path of the battery pack for current measurement.

HRS 06 – The hardware shall allow the BQ76952 to measure the voltage across the shunt resistor.

HRS 07 – The hardware shall support current measurement during charging and discharging.

HRS 08 – The hardware shall include temperature sensing components to monitor battery pack temperature. [I believe the BQ7695 does the temperature sensing here? Correct me if im wrong.](#)

HRS 09 – The hardware shall include I2C connections between the microcontroller and the BQ76952. [If so you wanna mention the hardware doing the sensing. Even if it is a different sensor breakout board you wanna write what is the Name of the sensor.](#)

HRS 10 – The hardware shall include pull-up resistors on the I2C communication lines if required by the circuit design.

HRS 11 – The hardware shall include protection circuitry for unsafe battery conditions such as overvoltage, undervoltage, overcurrent, and short circuit events. [talk about the exact elements for this circuit](#)

HRS 12 – The hardware shall support high-side NFET control for battery pack protection.

HRS 13 – The hardware shall allow the BQ76952 to disconnect or protect the battery pack during critical fault conditions.

HRS 14 – The hardware shall support cell balancing connections for reducing voltage differences between cells. [What hardware parts are involved in creating a cell balancing condition how does it work. Think what does this ? How does it do it?](#)

HRS 15 – The hardware shall be powered from the battery pack or an appropriate regulated supply.

HRS 16 – The hardware shall include appropriate connectors for the battery cells, microcontroller, communication lines, and measurement points.

HRS 17 – The hardware shall not allow battery cells to be connected in a way that exceeds the supported voltage range of the BQ76952. [this is a power architecture concern that can be eliminated when we select the components.](#)

HRS 18 – The hardware shall not bypass the current-sense shunt when current measurement is required. [this is again a concern and not a hardware requirement.](#)

HRS 19 – The hardware should include test points for cell voltage, pack voltage, current sense, ground, and communication signals. [This is good!](#)

HRS 20 – The hardware should be designed so that battery monitoring and protection can continue without requiring constant user input. [good.](#)

User Persona

Persona: Marcus Reed

Name: Marcus Reed

Age: 34

Gender: Male

Education: Bachelor's Degree in Mechanical Engineering

Location: Rochester, NY

Living Situation: Lives with his wife and young daughter in a single-family home

Occupation: Electric bike repair technician and small business owner

Technical Experience: Comfortable with basic electrical systems, battery packs, multimeters, and repair tools, but not an expert in embedded software or battery monitoring ICs

Hobbies: Riding electric bikes, repairing electronics, camping, and teaching his daughter how things work

Favorite Brands: Bosch, Anker, Milwaukee, Samsung

Devices Used: Electric bike, portable power station, laptop, smartphone, and digital multimeter

Background

Marcus owns a small electric bike repair shop. Many of his customers bring in battery packs that are not charging correctly, losing range too quickly, or shutting off during use. Since battery problems can be dangerous, Marcus needs a reliable way to monitor battery voltage, current, and temperature before deciding whether a battery pack is safe to use.

Marcus is not designing battery management hardware from scratch, but he understands why battery safety matters. He wants a device that can clearly show whether a battery pack is healthy or unsafe. He does not want to guess based only on how the battery "feels" or how long it lasts during a ride.

Goals

Marcus wants to use the Battery Management System to:

- Monitor individual battery cell voltages
- Detect unsafe voltage, current, and temperature conditions
- Prevent battery damage during charging and discharging
- Identify weak or unbalanced cells
- Improve customer safety
- Reduce the risk of overheating, short circuits, or battery failure

Frustrations

Marcus gets frustrated when battery systems do not clearly explain what is wrong. Some battery packs simply stop working without showing whether the issue is low voltage, overcurrent, overheating, or cell imbalance. He also dislikes systems that require too much technical setup before they can provide useful information.

His biggest frustration is that battery failures can become dangerous quickly. If a battery pack overheats or a cell voltage becomes too high or too low, the user may not notice until the damage has already happened.

Needs

Marcus needs the device to be:

- Safe and reliable
- Easy to connect to a battery pack
- Accurate enough to detect small voltage changes
- Able to warn the user when a fault occurs
- Capable of monitoring multiple cells in series
- Useful for both testing and real battery operation

How Marcus Would Use the Device

Marcus would connect the Battery Management System to a rechargeable battery pack before testing or repairing it. The system would monitor each cell voltage, the total pack current, and the battery temperature. If the BMS detects an unsafe condition, such as overvoltage, undervoltage, overcurrent, or overheating, it would alert Marcus or trigger protection behavior.

For Marcus, the most important feature is trust. He needs to know that the system is constantly checking the battery and will respond if something becomes unsafe. This helps him protect his customers, his equipment, and his repair shop.

Design Takeaway

This persona shows that the BMS should not only be technically functional, but also understandable to someone who works with batteries in real situations. The device should provide clear fault information, accurate measurements, and dependable protection so users like Marcus can make safe decisions without needing to understand every internal detail of the BQ76952 or microcontroller firmware.

Readme.md

Device Name

Introduction - (explains what the product is and its importance)

User Persona

Background

Goals

Frustrations

etc...

Hardware Requirements

Overview

Definitions

Functionality

Table with HRS number and actual HRS

Software Requirements

Similar to HRS section

Simple System Block Diagram

image

Detailed System Block Diagram.